

분기 예측기가 없는 Cortex-M0+에서 branchless 최적화의 역전과 LLM 기반 컴파일러 한계 극복 방향

차주형

과학기술연합대학원대학교

e-mail : chacha@ust.ac.kr

When Branchless Code Loses: Measuring Compiler Limits on a Branch-Predictor-less Cortex-M0+ and an LLM-based Way Forward

JooYoung Cha

University of Science and Technology (UST)

I. 서론

Abstract

Branchless programming is common folklore for data-dependent operations such as sign-dependent variable shifts ($v \ll s$ if $s > 0$, $v \gg |s|$ otherwise). This paper examines whether the folklore holds on the Cortex-M0+, an in-order microcontroller core without branch predictors, by comparing a branch baseline, a branchless C implementation, and a hand-written assembly implementation on real RP2040 hardware. The branchless C version was $1.2\times$ slower than the plain branch, because the C standard forbids the compiler from exploiting the ARM barrel shifter’s saturation semantics (shift amounts ≥ 32 yield zero) and the generated code spends 14 instructions per element. A hand-written assembly loop that delegates sign selection to the shifter saturation needs only 8 instructions per element and runs $1.5\times$ faster than the branch baseline, with measured cycles matching the analytical model within 2%. We analyze the gap between compiler-generated and hand-written code, attribute it to language-rule constraints, destructive two-operand encoding, and register allocation, and discuss a direction in which large language models predict such compiler-pass limitations and synthesize new assembly kernels or compiler passes, as demonstrated by the LLM-driven feedback loop used throughout this study.

부호에 따라 시프트 방향이 바뀌는 연산

$$f(v, s) = \begin{cases} v \ll s & (s > 0) \\ v \gg |s| & (s \leq 0) \end{cases} \quad (1)$$

은 고정소수점 신호처리나 양자화 신경망의 스케일링에서 자주 나타난다. 이 연산의 직관적인 구현은 부호 검사 분기를 포함하는데, 시프트 방향이 무작위에 가까우면 분기 예측이 불가능하므로 “분기를 제거하라(branchless)”는 것이 널리 알려진 최적화 상식이다[1, 2].

그러나 이 상식은 분기 예측 실패가 비싸다는 전제 위에서 있다. 사물인터넷 기기에 널리 쓰이는 Cortex-M0+[3]는 분기 예측기가 아예 없고 분기 비용이 1-3사이클로 고정된 프로세서다. 이런 코어에서도 분기 제거가 이득일까? 그리고 컴파일러에 최적화를 맡기는 것과 사람이 어셈블리를 직접 작성하는 것 사이에는 얼마나 차이가 있을까?

본 논문은 이 두 질문에 답하기 위해 RP2040 실제 하드웨어에서 세 가지 구현—분기 기준 구현(origin), 분기 제거 C 구현(shift_pow2), 수동 어셈블리 구현(sum_dual_asm)—의 성능을 실측 비교한다. 실험 결과 분기 제거 C 구현은 오히려 기준보다 1.2배 느렸고, 동일한 분기 제거 아이디어를 어셈블리로 직접 작성하면 기준보다 1.5배 빨랐다. 우리는 컴파일러가 생성한 실제 코드를 분석하여 이 역전의 원인이 컴파일러의 능력 부족이 아니라 언어

규칙과 명령어 집합 구조가 만드는 구조적 한계임을 보이고, 이러한 한계를 대형 언어 모델(LLM)로 예측하고 보완하는 방향을 제시한다.

II. 배경: Cortex-M0+의 세 가지 특성

실험 플랫폼인 RP2040은 125 MHz 듀얼 Cortex-M0+와 264 KB SRAM으로 구성된 마이크로컨트롤러다[4]. 본 연구와 관련된 M0+의 특성은 세 가지다.

첫째, **분기가 싸다**. 2단 인오더 파이프라인이라 분기 예측기가 없으며, 분기는 방향과 무관하게 1~3사이클이다. 데스크톱 프로세서에서 15사이클 이상인 예측 실패 비용이 여기에는 존재하지 않는다.

둘째, **실행 시간이 결정적이다**. 대부분의 명령어가 1사이클, 로드가 2사이클로 고정되어[3], 성능이 명령어 수로 직접 환산된다. 즉 “몇 명령어로 짰는가”가 곧 성능이다.

셋째, **배럴 시프터에 포화 특성이 있다**. 레지스터로 시프트 양을 지정하는 명령어(LSLS Rd, Rm)는 Rm의 하위 8비트를 시프트 양으로 사용하고, 그 값이 32 이상이면 결과를 0으로 만든다[5]. 뒤에서 보듯 이 특성은 부호 판단을 하드웨어에 공짜로 떠넘길 수 있게 해 주지만, C 표준이 워드 폭 이상의 시프트를 미정의 동작(undefined behavior)으로 규정하기 때문에 컴파일러는 이를 활용하는 코드를 생성할 수 없다.

III. 세 가지 구현

세 구현 모두 값 32에 대해 50,000개의 시프트 지시자(양수 1~20, 음수 -1~-20 반반)를 처리하며, 누적 합의 일치로 정합성을 검증하였다.

3.1 origin: 분기 기준 구현

식 (1)를 그대로 옮긴 C 코드다. 부호를 검사하는 조건 분기 하나와 시프트 하나로 구성되며, 이후 모든 비교의 기준이 된다.

3.2 shift_pow2: 분기 제거 C 구현

값이 $32 = 2^5$ 이므로 $f(32, s) = 2^{s+5}$ 로 두 경우를 한 식으로 통일할 수 있다. 지수 $e = s + 5$ 가 음수가 되는 경우($s < -5$, 결과 0)는 분기 대신 부호 마스크로 처리한다.

```
uint32_t shift_pow2(int8_t s) {
    int32_t e = (int32_t)s + 5;
    return (1u << ((uint32_t)e & 31))
        & ~(uint32_t)(e >> 31);
}
```

표 1: 50,000원소 처리 성능 (원소당 평균)

구현	ns/op	사이클	기준 대비
origin (분기)	120.2	15.0	1.00×
shift_pow2 (branchless C)	144.3	18.0	0.83×
sum_dual_asm	80.2	10.0	1.50×

분기가 완전히 사라진 전형적인 branchless 코드지만, 그 대가로 마스크 생성과 적용에 ALU 연산이 추가된다.

3.3 sum_dual_asm: 수동 어셈블리 구현

같은 분기 제거를 산술 연산이 아니라 **배럴 시프터의 포화 특성**으로 달성한다. $n = (\text{uint8}) s$ 로 두면 2의 보수 표현에 의해 음수 s 는 $n \in [236, 255]$ 가 되므로,

$$f(v, s) = (v \ll n) + (v \gg ((-n) \& 0xFF)) \quad (2)$$

에서 부호에 따라 한쪽 항이 하드웨어에 의해 자동으로 0이 된다. 비교도 마스크도 필요 없다. 그러나 2장에서 언급했듯 이 식은 C로 표현하면 미정의 동작이므로, 루프 전체를 어셈블리로 작성하였다. 원소당 8명령어이며, 4배 연돌과 수동 레지스터 할당(low 레지스터 6개)을 적용했다.

```
ldrb r3, [p, #k]    @ n=(uint8)s
movs r4, #32
lsls r4, r4, r3      @ n>=32 -> 0
negs r3, r3
movs r5, #32
lsrs r5, r5, r3      @ (-n)>=32 -> 0
adds acc, acc, r4
adds acc, acc, r5
```

IV. 실험 결과 및 분석

4.1 실험 환경 및 결과

Arduino Nano RP2040 Connect(125 MHz), arm-none-eabi-GCC 7.2(-O3)를 사용하였고, 시간은 1 MHz 하드웨어 타이머로 측정하였다. 표 1은 원소당 평균 처리 시간이다.

분기 제거 C 구현은 기준보다 오히려 1.2배 느렸고, 같은 목적의 수동 어셈블리는 1.5배 빨랐다. 원인을 확인하기 위해 컴파일러가 실제로 생성한 코드를 분석하였다.

4.2 컴파일러 생성 코드와의 차이 분석

GCC -O3가 shift_pow2 루프에 생성한 실제 코드는 원소당 14명령어다(루프 제어 포함).

```

movs r3,#0      @ zero offset
movs r4,r6      @ copy const 31
movs r7,r5      @ copy const 1
ldrsb r3,[r2,r3]
adds r2,r2,#1
adds r3,r3,#5    @ e = s+5
ands r4,r3      @ e & 31
lsls r7,r7,r4    @ 1 << (e&31)
movs r4,r7      @ copy result
asrs r3,r3,#31   @ sign mask
bics r4,r3      @ apply mask
adds r0,r0,r4
cmp r1,r2
bne .L3

```

수동 어셈블리(8명령어)와의 차이는 세 층위로 나뉜다.

첫째, **언어 규칙에 의한 알고리즘 제약**이다. 포화 특성을 쓰면 부호 처리가 0명령어인데, C의 미정의 동작 규칙 때문에 컴파일러는 이 변환 자체가 금지되어 있다. 그래서 **branchless C**는 마스크 생성-적용(`asrs, bics`)과 시프트 양 클램프(`ands`)라는 우회 경로를 강제당한다. 이것은 최적화기의 능력 문제가 아니라 규칙 문제다.

둘째, **명령어 집합 구조에 의한 낭비**다. Thumb-1은 대부분 2 피연산자 파괴형 명령어라 상수(1, 31)를 보존하려면 매 반복 레지스터 복사(`movs r4, r6` 등)가 필요하다. 위 코드에서 원소당 3개의 `movs`가 순수한 이동 비용이다.

셋째, **이론과 실측의 오차가 작다**는 점이다. 수동 어셈블리의 이론 사이클은 명령어 8개 + 로드 추가 1 + 루프 분담 $1.25 = 10.25$ 인데 실측은 10.0으로 오차 2% 이내다. M0+의 결정적 실행 모델에서는 명령어 수 분석만으로 성능을 정확히 예측할 수 있으며, 반대로 말하면 컴파일러가 만든 14명령어와 사람이 만든 8명령어의 차이가 그대로 성능 차(18 대 10사이클)로 나타난다.

한편 동일한 어셈블리 아이디어라도 조각(fragment) 단위로 C 루프에 삽입하면 기준과 동률 (120.2 ns)에 그쳤다. 컴파일러는 인라인 어셈블리 내부를 볼 수 없어 조각마다 레지스터 4개를 통째로 예약하고, -O3 언롤 과정에서 8개뿐인 low 레지스터가 고갈되어 스택 스푼이 발생하기 때문이다. 어셈블리 최적화는 조각이 아니라 루프 단위로 적용해야 효과가 있다.

4.3 논의 및 한계

이 결과에서 세 가지 교훈을 끌어낼 수 있다. 첫째, 분기 예측기가 없는 코어에서 분기는 이미 1-3사이클로 저렴하므로, 분기 하나를 없애기 위해 ALU 연산을 네 개 이상 추가하는 변환은 손해다. **branchless**가 유리하다는 상식은 마이크로아키텍처에 따라

역전된다. 둘째, 하드웨어가 제공하는 유용한 동작(시프트 포화)이 언어 표준에서는 미정의로 묶여 있을 때, 그 간극은 컴파일러 버전이 올라가도 좁혀지지 않는다. 규칙의 문제이지 구현의 문제가 아니기 때문이다. 셋째, 결정적 실행 코어에서는 종이 위의 사이클 계산이 실측과 2% 이내로 맞으므로, 최적화 여지가 있는지 여부를 코드를 실행하기 전에 판정할 수 있다.

본 실험의 한계도 명확히 해 둔다. 수동 어셈블리 구현은 값이 상수 32라는 조건과 시프트 지시자가 ± 31 이내라는 조건을 전제한다. 값이 배열로 주어지면 상수 적재(`movs`)가 로드(`ldr`)로 바뀌어 원소당 2-3사이클이 추가되지만 포화 트릭 자체는 그대로 유효하다. 또한 본 결과는 분기 예측기가 있는 상위 코어(Cortex-M7 등)로 일반화할 수 없으며, 오히려 그런 코어에서는 **branchless**가 다시 유리해질 수 있다. 즉 최적 구현은 코어마다 다시 실측되어야 하고, 이것이 다음 장에서 논의할 자동화가 필요한 이유다.

V. LLM을 이용한 컴파일러 한계 극복

4장의 분석은 컴파일러 패스가 넘을 수 없는 간극이 실재함을 보인다. 미정의 동작 뒤에 숨은 하드웨어 의미론(시프트 포화), 대상 코어의 사이클 모델, 레지스터 압박과 언롤의 상호작용은 각각 프런트엔드, 백엔드, 할당기에 흩어져 있어 단일 패스가 종합적으로 추론하기 어렵다.

본 연구는 이 간극을 LLM으로 메우는 가능성을 실증한다. 실제로 본 논문의 `sum.dual.asm`은 LLM이 주도하는 반복 피드백 루프—실기 측정, 병목 분석, 코드 수정, 재측정—를 통해 도출되었다. 이 과정에서 LLM은 (1) ISA 문서의 포화 의미론과 C 표준의 미정의 동작 규칙을 대조하여 “컴파일러가 규칙상 생성할 수 없는 최적 코드”의 존재를 예측하였고, (2) M0+ 사이클 모델로 이론 성능(10.25사이클)을 사전 계산해 실측(10.0)과 2% 이내로 일치시켰으며, (3) 어셈블리 조각 방식의 실패(레지스터 스푼)를 분석해 루프 단위 재작성으로 전환하였다. 정합성은 매 반복 하드웨어에서 기준 구현과의 결과 비교로 검증되므로, LLM의 생성 오류는 루프 안에서 걸러진다.

이 접근이 성립하는 조건도 실험에서 드러난다. LLM이 제안한 첫 어셈블리는 어셈블리 문법 불일치로 두 차례 컴파일에 실패했고, 어셈블리 조각 방식은 성능 목표에 미달했다. 그럼에도 최종 구현에 도달할 수 있었던 것은 컴파일 오류, 실측 성능, 정합성 검증이라는 세 겹의 객관적 피드백이 매 반복 제공되었기 때문이다. 즉 LLM 기반 최적화의 신뢰성은 모델 단독의 정확성이 아니라 하드웨어가 포함된 페루프 구조에서 나온다.

이는 두 방향으로 확장될 수 있다. 첫째, LLM을 **패스 한계 예측기**로 사용하여, 주어진 커널과 대상 코어에 대해 “컴파일러 출력

과 이론 하한의 격차”를 자동 추정하고 수동 최적화가 필요한 지점만 선별하는 것이다. 둘째, LLM이 발견한 변환—예컨대 부호 가변 시프트를 식 (2)의 포화 이중 시프트로 바꾸는 것—을 페리홀 (peephole) 패스나 명령어 선택 패턴으로 형식화하여 컴파일러에 환류하는 것이다. 전자는 개발자 노력의 배분을, 후자는 발견된 지식의 재사용을 자동화한다. 하드웨어 실측이 페루프에 포함되어 있다는 점에서, 이 접근은 정확성 보장이 어려운 LLM 단독 코드 생성과 구별된다.

VI. 결론

본 논문은 분기 예측기가 없는 Cortex-M0+에서 부호 가변 시프트 연산의 세 구현을 실측 비교하였다. 분기 제거 C 구현은 기준 분기 구현보다 1.2배 느려 “branchless가 빠르다”는 상식이 역전 되었고, 배럴 시프트의 포화 특성을 이용한 수동 어셈블리는 1.5배 빨랐다. 컴파일러 생성 코드 분석을 통해 이 차이가 언어 규칙(미정의 동작), 명령어 집합 구조(2피연산자 파괴형), 레지스터 할당의 세 층위에서 비롯됨을 보였으며, 이러한 컴파일러 패스의 구조적 한계를 LLM 기반 피드백 루프로 예측하고 보완할 수 있음을 본 연구의 수행 과정 자체로 실증하였다. 향후 다양한 커널과 코어로 실험을 확장하고, LLM이 발견한 변환을 컴파일러 패스로 형식화하는 자동화 도구를 연구할 예정이다.

참고 문헌

- [1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th ed., 2017.
- [2] Piorosen, “Shiftoperator: Branch-free shift via self-modifying code.” <https://github.com/Piorosen/ShiftOperator>. Accessed 2026-07-20.
- [3] Arm Ltd., *Cortex-M0+ Technical Reference Manual*, 2012. DDI 0484C.
- [4] Raspberry Pi Ltd., *RP2040 Datasheet*, 2024.
- [5] Arm Ltd., *ARMv6-M Architecture Reference Manual*, 2018. DDI 0419E.